Image Classification of Brain Tumors

Project Report

1. Introduction

For our Machine Learning project, we decided to focus on tumor detection using image classification. When picking a topic, we wanted to choose something meaningful and interesting. Since medical datasets are quite accessible and neural networks are being widely used in this area, we decided to go with the healthcare field, and tumor detection was the direction that caught our attention the most.

Detecting the presence of tumors early and accurately is really important in medicine, because it can have a big impact on treatment choices and patient outcomes. Doctors often examine medical images to determine whether a tumor is present, but this process can be time-consuming and challenging due to subtle variations in image appearance. Machine learning, especially image classification, can help by providing automated and more consistent analysis of tumor images [**?**].

The report is structured as follows: Section 1 introduces the task and motivation; Section 2 formulates the problem, describing the data, labels, and setup; Section 3 outlines the methodology, including preprocessing, CNN model, loss/optimizer choices, and data splits.

2. Problem formulation

Our application can be formalized as a supervised binary classification problem. The goal is to classify MRI images of brains as either containing a tumor or not. Each MRI scan works as a data point, and the features are the pixel values of the image. To make sense of these pixels and find useful patterns, we use Convolutional Neural Networks (CNNs).

The dataset we are using for this project consists of MRI (Magnetic Resonance Imaging) scans, which are images of brains with and without tumors. The images are publicly available on Kaggle, provided by Preetviradiya [**?**]. It is a simple binary dataset with two classes: ”Brain Tumor” and ”Healthy”. The label for each image is binary, meaning it just tells us if a tumor is present or not. Below, you can see an example of an image from the dataset:

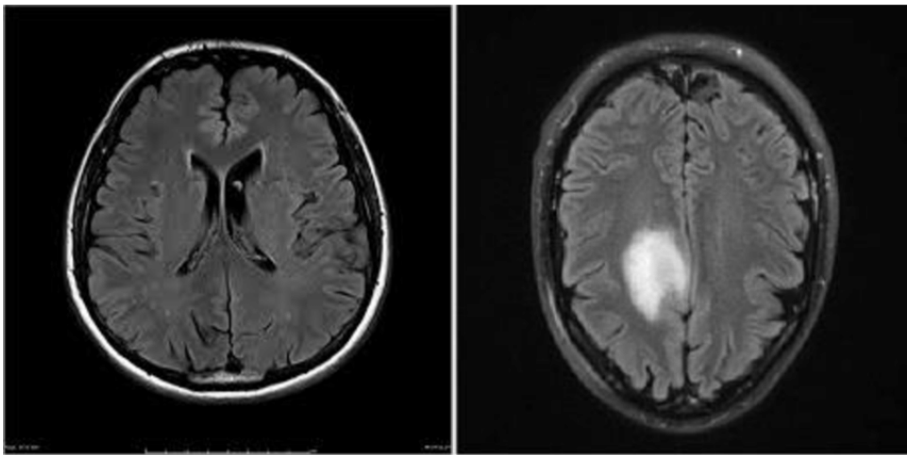


Figure 1: Example of a brain MRI scan with and without a tumor from the dataset.

For the implementation and experiments, we use Python with Jupyter Notebook as the development

environment and TensorFlow as the main deep learning framework.

3. Methods

We started with splitting the dataset into training, validation, and test sets using a 70% / 20% / 10% ratio. The training set has 2343 brain tumor images and 2066 healthy images, the validation set has 1406 brain tumor images and 1237 healthy images, and the test set has 811 brain tumor images and 724 healthy images. We chose this ratio for splitting the dataset because it is commonly used in machine learning projects and provides a good balance between training, validation, and testing.

The dataset is relatively balanced, with only about a 6% difference between the number of samples in each class. This small imbalance should not pose a problem for our experiments. The slight imbalance can also be seen in the distribution chart (Figure 2).

Figure 2: Class distribution

The images in our dataset come in different sizes (Figure 1), so as part of preprocessing we resized them to 128×128 pixels using TensorFlow's ImageDataGenerator[**?**]. For training and validation, we also normalized the images to [0,1] and applied simple augmentations like horizontal flips, small zooms, and slight shearing to make the model more robust. The test images were only normalized to allow fair evaluation. All images were treated as RGB, categorized for binary classification (tumor vs. healthy), and the training set was shuffled to help generalization.

We chose a Convolutional Neural Network (CNN) because it works really well with images. CNNs can pick up important visual patterns like edges or textures that simpler models, such as linear classifiers or Support Vector Machines, would probably miss. While other methods can also be used for images, CNNs are generally the most effective and common choice for image classification tasks like ours.

As our loss function, we chose binary cross-entropy. We picked it because our task is a binary classification problem – the model predicts whether a brain MRI contains a tumor or not. Binary cross-entropy works well here because it measures how close the predicted probabilities are to the true labels (0 or 1).

For optimization, we used the Adam optimizer with a learning rate of 0.001. This optimizer is convenient because it automatically adjusts the learning rate for each parameter, which usually helps the model learn faster and more reliably than a regular gradient descent.

4. Results

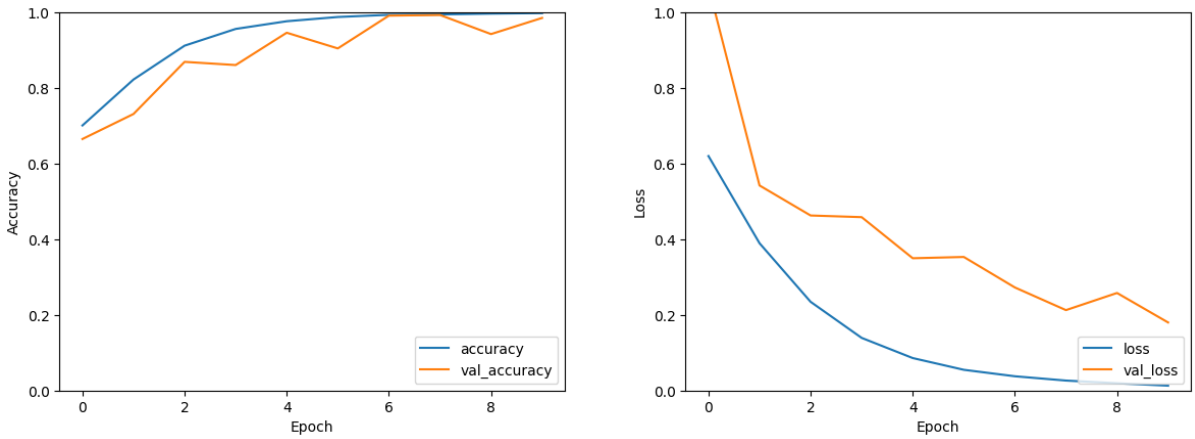
This section compares the performance of the two models trained for binary tumor classification: (1) a Convolution Neural Network (CNN) and (2) a simple linear Artificial Neural Network (ANN).

The CNN model consisted of two convolutional layers (with 32 and 64 filters, respectively), each followed by a MaxPooling layer. The convolutional base was flattened and passed through two fully connected layers with ReLU activation, batch normalization, and dropout for regularization, before reaching the final sigmoid output layer. The model was trained for 10 epochs using the Adam optimizer with a learning rate of 0.001.

At the end of training, the model achieved a **training accuracy of 99.88%**, a **validation accuracy of 98.56%**, and a **testing accuracy of 98.49%**. The corresponding losses were 0.0134 (training), 0.1800 (validation), and 0.1954 (testing).

Table 1: Summary of training, validation and test results (loss).

Model	Training Loss	Validation Loss	Test Loss
CNN	0.0134	0.1800	0.1954



(a) CNN: accuracy vs. epochs

(b) CNN: loss vs. epochs

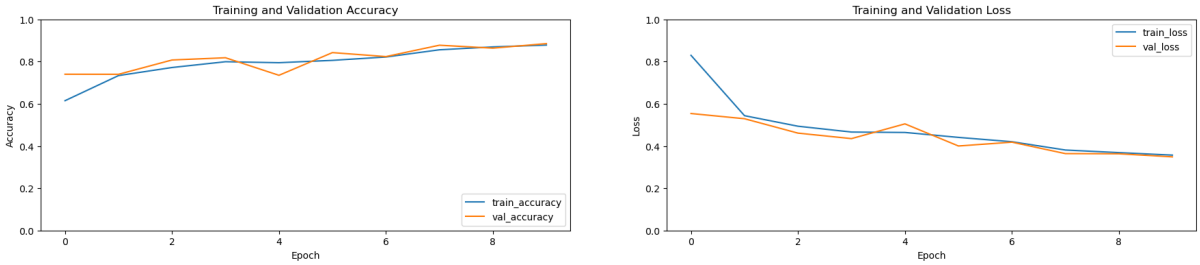
Figure 3: Training/validation curves for the CNN model

The second model was a simple feedforward ANN that flattened the input image and passed it directly to a sigmoid output neuron, effectively functioning as a linear classifier over pixel intensities.

After 10 epochs, the ANN achieved a **training accuracy of 87.83%**, a **validation accuracy of 88.52%**, and a **testing accuracy of 87.74%**. The losses were 0.357 (training), 0.3488 (validation), and 0.3481 (testing).

Table 2: Summary of training, validation and test results (loss).

Model	Training Loss	Validation Loss	Test Loss
Linear ANN	0.3570	0.3488	0.3481



(a) ANN: accuracy vs. epochs

(b) ANN: loss vs. epochs

Figure 4: Training/validation curves for the Linear ANN model.

5. Model Comparison and Final Selection

Comparing both architectures, it is evident that the CNN model significantly outperformed the linear ANN in terms of accuracy, loss, and overall generalization. The CNN model's ability to capture spatial hierarchies and local features allowed it to detect subtle differences in brain MRI images, which the linear ANN failed to model effectively.

The final chosen method is therefore the **CNN architecture**, due to its superior accuracy, lower error, and better generalization across all datasets. The **test error** of the final chosen method (CNN) is approximately **1.51%**, confirming its reliability for tumor classification.

To further assess the CNN's performance, we tabulated the precision-recall-F1 summary.

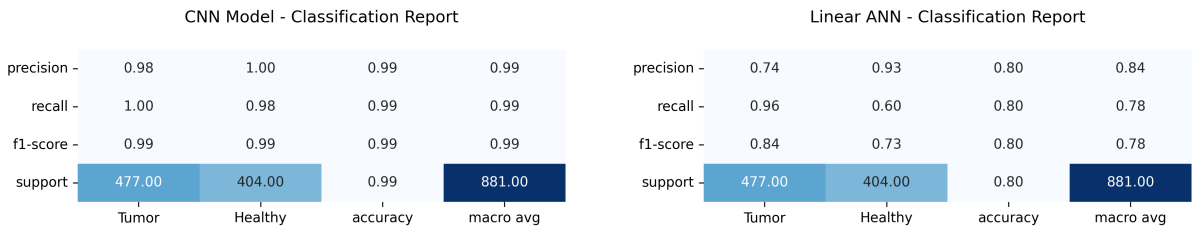


Figure 5: Classification report comparison between CNN and Linear ANN models.

6. Conclusion

This project demonstrated the successful application of deep learning to the problem of brain tumor classification from MRI images. The CNN achieved nearly 98.5% accuracy on the test set, suggesting that the model has learned to generalize well to unseen data. These results indicate that the problem has been solved satisfactorily for the current dataset. However, there remains room for improvement. The model was trained for only 10 epochs; extending training with early stopping or learning rate scheduling might yield even better generalization.

Limitations include the relatively small dataset size and potential overfitting despite dropout and normalization. Moreover, since medical imaging can vary across scanners and patient populations, testing the model on more diverse datasets would strengthen its reliability for real-world deployment.

References

[1] AB Abdusalomov, M Mukhiddinov, and TK Whangbo. Brain tumor detection based on deep learning approaches and magnetic resonance imaging. *Cancers (Basel)*, 15(16):4172, Aug 18 2023.

[2] Preetviradiya. Brain tumor dataset. `https://www.kaggle.com/datasets/preetviradiya/brian-tumor-dataset`, 2021. Accessed: 2025-09-16.

[3] TensorFlow Authors. ImageDataGenerator. `https://www.tensorflow.org/api_docs/python/tf/keras/preprocessing/image/ImageDataGenerator`, 2023. Accessed: 2025-09-17.

Appendix: Code

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import random
import seaborn as sns
import os
import shutil
from PIL import Image

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, Activation
from tensorflow.keras.layers import MaxPooling2D
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import SeparableConv2D
from tensorflow.keras.layers import BatchNormalization
from tensorflow.keras.layers import Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img
from tensorflow.keras.callbacks import ModelCheckpoint
from tensorflow.keras import optimizers
from tensorflow.keras import layers
from keras import regularizers
```

```python
#Check number of classes in Brain tumor directory
base_dir = 'Brain Tumor Data Set/Brain Tumor Data Set'
# Replace with your actual path
def count_classes(directory):
    return len([name for name in os.listdir(directory) if os.path.isdir(os.path.join(directory, name))])

classes = count_classes(base_dir)

print(f"Number of classes in the dataset : {classes}")
```

```python
# Set the directory paths for Tumor and Healthy images
Tumor = "Brain Tumor Data Set/Brain Tumor Data Set/Brain Tumor"
Healthy = "Brain Tumor Data Set/Brain Tumor Data Set/Healthy"
# Defined paths for train, validation, and test directories
train_dir = 'Brain Tumor Data Set/train'
val_dir = 'Brain Tumor Data Set/val'
test_dir = 'Brain Tumor Data Set/test'

 # Create directories for train, val, and test with subfolders for Brain Tumor and Healthy
os.makedirs(os.path.join(train_dir, 'Brain Tumor'), exist_ok=True)
os.makedirs(os.path.join(train_dir, 'Healthy'), exist_ok=True)
os.makedirs(os.path.join(val_dir, 'Brain Tumor'), exist_ok=True)
os.makedirs(os.path.join(val_dir, 'Healthy'), exist_ok=True)
os.makedirs(os.path.join(test_dir, 'Brain Tumor'), exist_ok=True)
os.makedirs(os.path.join(test_dir, 'Healthy'), exist_ok=True)
```

```python

 # Function to split dataset into train, val, and test sets
def split_data(src_dir, dest_train, dest_val, dest_test, train_pct=0.7, val_pct=0.2, test_pct=0.1):
     # Handle both Tumor and Healthy categories separately
    for category, category_dir in zip(['Brain Tumor', 'Healthy'], [Tumor, Healthy]):
         # Get all files in the category folder
        files = os.listdir(category_dir)
        random.shuffle(files)  # Shuffle the files

         # Calculate the number of files for each split
        total_files = len(files)
        train_count = int(total_files * train_pct)
        val_count = int(total_files * val_pct)
        test_count = total_files - train_count - val_count  # Remaining files go to test

     # Ensure the sum of counts does not exceed total files
        if train_count + val_count + test_count != total_files:
            test_count = total_files - train_count - val_count

         # Split the files into training, validation, and testing
        for i, file in enumerate(files):
            src_file = os.path.join(category_dir, file)

             # Move to the appropriate folder (train/val/test) based on split
            if i < train_count:
                shutil.copy(src_file, os.path.join(dest_train, category, file))
            elif i < train_count + val_count:
                shutil.copy(src_file, os.path.join(dest_val, category, file))
            else:
                shutil.copy(src_file, os.path.join(dest_test, category, file))

 # Split both the Tumor and Healthy data
split_data(Tumor, train_dir, val_dir, test_dir)
split_data(Healthy, train_dir, val_dir, test_dir)

print("Data split completed!")
```

```python
# Function to count the number of images in a directory
def count_images(directory):
    return len([file for file in os.listdir(directory) if file.endswith(('png', 'jpg', 'jpeg'))])

# Count the number of images in each subfolder of the 'Train' directory
brain_tumor_images_train = count_images(os.path.join(train_dir, 'Brain Tumor'))
healthy_images_train = count_images(os.path.join(train_dir, 'Healthy'))
brain_tumor_images_val = count_images(os.path.join(val_dir, 'Brain Tumor'))
healthy_images_val = count_images(os.path.join(val_dir, 'Healthy'))
brain_tumor_images_test = count_images(os.path.join(test_dir, 'Brain Tumor'))
```

```python
healthy_images_test = count_images(os.path.join(test_dir, 'Healthy'))


# Print the results
print(f"Number of Brain Tumor images in train directory: {brain_tumor_images_train}")
print(f"Number of Healthy images in train directory: {healthy_images_train}")
print(f"Number of Brain Tumor images in validation directory: {brain_tumor_images_val}")
print(f"Number of Healthy images in validation directory: {healthy_images_val}")
print(f"Number of Brain Tumor images in test directory: {brain_tumor_images_test}")
print(f"Number of Healthy images in test directory: {healthy_images_test}")
```

```python
# Choose a random image from the Brain Tumor and Healthy directories
random_tumor_image = random.sample(os.listdir(Tumor),3)
random_healthy_image = random.sample(os.listdir(Healthy),3)

# Open the selected images
tumor_images = [Image.open(os.path.join(Tumor, image)) for image in random_tumor_image]
healthy_images = [Image.open(os.path.join(Healthy, image)) for image in random_healthy_image]

# Create a figure for displaying the images
figure = plt.figure(figsize=(10,15))

# Display the tumor images in the first column of subplots
for i in range(3):
    subplot = figure.add_subplot(3, 2, 2 * i + 1)
    plt.imshow(tumor_images[i])
    subplot.set_title(f"Tumor")

# Display the healthy images in the second column of subplots
for i in range(3):
    subplot = figure.add_subplot(3, 2, (2 * i) + 2)
    plt.imshow(healthy_images[i])
    subplot.set_title(f"Healthy")

# Adjust layout for better spacing
plt.tight_layout()

# Show the figure
plt.show()
```

```python
# Define the paths for the 'Brain Tumor' and 'Healthy' directories
train_Tumor = 'Brain Tumor Data Set/train/Brain Tumor'
train_Healthy = 'Brain Tumor Data Set/train/Healthy'

# Choose 3 random images from each directory
random_tumor_images = random.sample(os.listdir(train_Tumor), 3)  # 3 unique tumor images
random_healthy_images = random.sample(os.listdir(train_Healthy), 3)  # 3 unique healthy images
```

```python

# Function to get image shape
def get_image_shape(image_path):
    with Image.open(image_path) as img:
        img_array = np.array(img)  # Convert image to numpy array
        return img_array.shape  # Returns shape (height, width, channels)

# Print image shapes for Tumor images
print("Tumor images shapes:")
for image in random_tumor_images:
    image_path = os.path.join(Tumor, image)
    shape = get_image_shape(image_path)
    print(f"Image: {image} | Shape: {shape}")

# Print image shapes for Healthy images
print("\nHealthy images shapes:")
for image in random_healthy_images:
    image_path = os.path.join(Healthy, image)
    shape = get_image_shape(image_path)
    print(f"Image: {image} | Shape: {shape}")
```

```python
#joining both directories into training data and testing data and validation data
gen = ImageDataGenerator(shear_range=0.2,horizontal_flip=True,rescale=1./255, zoom_range=(0.99,0.99)) #will be used for both training and validation data
test_gen = ImageDataGenerator(rescale=1.0/255)

# Training data
train = gen.flow_from_directory("Brain Tumor Data Set/train",
                                target_size = (128,128),
                                batch_size = 256,
                                color_mode = "rgb",
                                class_mode = "binary",
                                shuffle = True)

#Validation data
val = gen.flow_from_directory("Brain Tumor Data Set/val",
                                target_size = (128,128),
                                batch_size = 256,
                                color_mode = "rgb",
                                class_mode = "binary",
                                shuffle = True)
test = test_gen.flow_from_directory("Brain Tumor Data Set/test",
                                target_size = (128,128),
                                color_mode = "rgb",
                                class_mode = "binary",
                                shuffle = False)
train.image_shape
```

```python
# Get the labels from the train and validation data generators
labels = train.labels
labels_val = val.labels

```

```python
# Create the subplots (1 row, 2 columns)
plt.figure(figsize=(8, 4))

# Plot the histogram for training data (subplot 1)
plt.subplot(1, 2, 1)
n, bins, patches = plt.hist(labels, bins=2, edgecolor='black', alpha=1, width=0.4)

# Set the labels for the x-axis (0: Tumor, 1: Healthy) at the center of each bar
bar_centers = (bins[:-1] + bins[1:]) / 2  # Calculate the center of each bar
plt.xticks(bar_centers, ['Tumor', 'Healthy'])

# Set the title and axis labels for the training data histogram
plt.title('Class Distribution in Training Data')
plt.xlabel('Class')
plt.ylabel('Frequency')

# Plot the histogram for validation data (subplot 2)
plt.subplot(1, 2, 2)
n_val, bins_val, patches_val = plt.hist(labels_val, bins=2, edgecolor='black', alpha=1, width=0.4)

# Set the labels for the x-axis (0: Tumor, 1: Healthy) at the center of each bar for validation data
bar_centers_val = (bins_val[:-1] + bins_val[1:]) / 2  # Calculate the center of each bar
plt.xticks(bar_centers_val, ['Tumor', 'Healthy'])

# Set the title and axis labels for the validation data histogram
plt.title('Class Distribution in Validation Data')
plt.xlabel('Class')
plt.ylabel('Frequency')

# Show the histograms
plt.tight_layout()  # Adjust layout for better spacing
plt.show()
```

```python
# CNN Model
model = Sequential()
model.add(layers.InputLayer(input_shape=(128, 128, 3)))
model.add(layers.Conv2D(filters=32, kernel_size=3, activation="relu", padding="same"))
model.add(layers.MaxPooling2D())
model.add(layers.Conv2D(filters=64, kernel_size=3, activation="relu", padding="same"))
model.add(layers.MaxPooling2D())
model.add(layers.Flatten())
model.add(layers.Dense(128, activation='relu'))
model.add(layers.BatchNormalization())
model.add(layers.Dropout(rate=0.3))
model.add(layers.Dense(64, activation="relu"))
model.add(layers.BatchNormalization())
model.add(layers.Dropout(rate=0.3))
model.add(layers.Dense(1, activation="sigmoid"))
```

```python
# Define the EarlyStopping callback
```

```
from keras import utils, callbacks
earlystopping = callbacks.EarlyStopping(monitor="val_loss", mode="min",
                                        patience=5, restore_best_weights = True)
```

```
# Compile the model with Adam optimizer
adam_optimizer = optimizers.Adam(learning_rate=0.001)
model.compile(optimizer=adam_optimizer,loss="binary_crossentropy", metrics=['accuracy'])
```

```
#Fit the model
history = model.fit(train,verbose=1,epochs=10,validation_data=val,callbacks = [earlystop
```

```
#evaluating the model on test data
test_accuracy = model.evaluate(test),
test_accuracy
```

```
#Accuracy and Loss Curves
plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label = 'val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0, 1])
plt.legend(loc='lower right')

plt.plot(history.history['loss'], label='loss')
plt.plot(history.history['val_loss'], label = 'val_loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.ylim([0, 1])
plt.legend(loc='lower right')
```

```
#Saving Model
model.save('model.keras')
```

```
# Define a simple linear model with one hidden layer
model2 = Sequential([
    Input(shape=(128,128,3)),
    Flatten(),
    #Dense(128, activation='relu'),
    Dense(1, activation='sigmoid')
])
# Compile the model
model2.compile(
    optimizer=Adam(0.001),
    loss='binary_crossentropy',
    metrics=['accuracy']
)
# Train the model
history = model2.fit(
```

```
    train,
    validation_data=val,
    epochs=10
)
```

```
# Plot Loss
plt.figure(figsize=(10,4))
plt.plot(history.history['loss'], label='train_loss')
plt.plot(history.history['val_loss'], label='val_loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.ylim([0, 1])
plt.title('Training and Validation Loss')
plt.legend(loc='upper right')
plt.show()

# Plot Accuracy
plt.figure(figsize=(10,4))
plt.plot(history.history['accuracy'], label='train_accuracy')
plt.plot(history.history['val_accuracy'], label='val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0, 1])
plt.title('Training and Validation Accuracy')
plt.legend(loc='lower right')
plt.show()
```

```
#Test evaluation
testing_accuracy = model2.evaluate(test)
```

```
#Confusion Matrix
# Make sure test generator has shuffle=False
test.reset()

# Predictions for model 1 (e.g., linear model)
y_pred1 = (model2.predict(test) > 0.5).astype(int)
y_true = test.labels
cm1 = confusion_matrix(y_true, y_pred1)

# Predictions for model 2 (e.g., CNN model)
y_pred2 = (model1.predict(test) > 0.5).astype(int)
cm2 = confusion_matrix(y_true, y_pred2)

# Plot both confusion matrices side by side
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

disp1 = ConfusionMatrixDisplay(confusion_matrix=cm1, display_labels=['Tumor', 'Healthy']
disp1.plot(ax=axes[0], cmap=plt.cm.Blues, colorbar=False)
axes[0].set_title('Linear Model')

disp2 = ConfusionMatrixDisplay(confusion_matrix=cm2, display_labels=['Tumor', 'Healthy']
disp2.plot(ax=axes[1], cmap=plt.cm.Oranges, colorbar=False)
axes[1].set_title('CNN Model')

```

```
plt.tight_layout()
plt.show()
```

```
#Classification Report
test.reset()

# Predictions for linear model
y_pred1 = (model2.predict(test) > 0.5).astype(int)
y_true = test.labels
print("=== Linear Model ===")
print("Accuracy:", accuracy_score(y_true, y_pred1))
print(classification_report(y_true, y_pred1, target_names=['Tumor', 'Healthy']))

# Predictions for CNN model
y_pred2 = (model1.predict(test) > 0.5).astype(int)
print("=== CNN Model ===")
print("Accuracy:", accuracy_score(y_true, y_pred2))
print(classification_report(y_true, y_pred2, target_names=['Tumor', 'Healthy']))
```

```
#Small Python Application for testing on random Images
import streamlit as st
import numpy as np
import tensorflow as tf
from PIL import Image

# ------------------------------
# Load TFLite model
# ------------------------------
@st.cache_resource  # cache so it's loaded only once
def load_tflite_model():
    interpreter = tf.lite.Interpreter(model_path="model.tflite")
    interpreter.allocate_tensors()
    return interpreter

interpreter = load_tflite_model()
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()

# ------------------------------
# Prediction function
# ------------------------------
def predict_tflite(image: Image.Image, target_size=(128, 128)):
    # Resize & preprocess
    img = image.resize(target_size)
    img_array = np.array(img).astype(np.float32) / 255.0
    img_array = np.expand_dims(img_array, axis=0)

    # Set input tensor
    interpreter.set_tensor(input_details[0]['index'], img_array)

    # Run inference
    interpreter.invoke()

    # Get prediction
    prediction = interpreter.get_tensor(output_details[0]['index'])
    prediction_value = prediction[0][0]
```

```python

    if prediction_value > 0.5:
        return f" Healthy ({prediction_value * 100:.2f}%)"
    else:
        return f" Tumor ({(1 - prediction_value) * 100:.2f}%)"

# -------------------------------
# Streamlit UI
# -------------------------------
st.title(" Brain Tumor Detection (TFLite Model)")
st.write("Upload an MRI scan to check if it shows **Healthy** or **Tumor**.")

uploaded_file = st.file_uploader("Upload an image", type=["jpg", "jpeg", "png"])

if uploaded_file is not None:
    # Display uploaded image
    img = Image.open(uploaded_file).convert("RGB")
    st.image(img, caption="Uploaded Image", use_container_width=True)

    # Make prediction
    result = predict_tflite(img)
    st.subheader("Prediction Result")
    st.success(result)
```